The Three-Point Revolution: A Quantitative Analysis of the NBA's Strategic Transformation

Adam Rychtecky

2026-06-03

DRAFT ABSTRACT — replace entirely with your own words before submission.

This project quantifies the NBA's three-point revolution across 4,222,442 shot attempts spanning 21 seasons (2004–2024), processed on Google Cloud using Apache Spark and MLlib. Three connected analyses address why the shift was mathematically inevitable, when it accelerated, and how it reorganized the player landscape. Zone efficiency analysis reveals that mid-range shots generate only 0.798 points per attempt versus 1.054 for above-the-break threes — a 32% efficiency gap on attempts with nearly identical field-goal percentages (39.9% vs. 35.1%). A LASSO-regularized shot-make classifier (ROC-AUC = 0.629) confirms that shot location and action type dominate game-context features in predicting whether a shot falls. League-wide three-point attempt rate rose from 18.6% in 2004 to 39.4% in 2024, with the sharpest two-year acceleration between 2015 and 2017. Unsupervised clustering of player shot profiles identifies five archetypes: the mid-range specialist archetype declined from 37.3% to 7.7% of qualifying player-seasons after 2015, while a modern balanced-scoring archetype expanded from 8.1% to 35.8%.

Table of contents

1 Introduction

2 Data

2.1 Source and Coverage

The primary dataset is NBA shot-location records for all 21 seasons from 2003–04 through 2024–25 (referred to throughout by the calendar year the season began, e.g., "2004" for 2003–04). Data were sourced from the publicly available NBA_Shots_04_25 repository (Samangy, 2025), which aggregates shot-level records from the official NBA Stats API (stats.nba.com). The raw dataset comprises 4,231,262 shot attempts across 26 columns, including court coordinates (`LOC_X`, `LOC_Y`), shot distance, zone classification (`BASIC_ZONE`), action type, shot type (2PT or 3PT), game context fields (quarter, minutes and seconds remaining), player and team identifiers, and a make/miss indicator.

Raw CSV files were uploaded to Google Cloud Storage (bucket: `pstat135-adam`) and ingested using Apache Spark on a Google Cloud Dataproc cluster (single-node `e2-highmem-4`, `us-central1`, Dataproc image 2.2-debian12). Spark's schema inference confirmed the 26-column structure and zero null values across all fields.

2.2 Cleaning and Feature Engineering

Data preparation removed 8,820 backcourt shots (desperation heaves recorded with `ZONE_RANGE = "Back Court Shot"`) that do not represent genuine shooting decisions, yielding **4,222,442 analysis-ready records**. Seven feature columns were added: `SECS_REMAINING` (clock context as a single integer), `IS_3PT` and `SHOT_VALUE` (two/three-point indicators), `IS_CLUTCH` (fourth quarter or overtime with 120 seconds remaining), `SHOT_MADE_INT` (integer cast of the boolean target, required for Spark MLlib), `GAME_DATE_PARSED`, and `EXPECTED_PTS` (`SHOT_VALUE × SHOT_MADE_INT`). The `ACTION_TYPE` field, which carries dozens of raw values, was consolidated into five broad buckets (Dunk, Layup, Hook, Jump Shot, Other) via keyword matching to prevent sparse one-hot expansion in model training; the residual "Other" category accounts for approximately 1.4% of shots.

The cleaned dataset was written to GCS as Parquet, partitioned by season (`SEASON_1`), enabling efficient season-level filtering in downstream analyses.

2.3 Distributed Processing Justification

A single season's records fit comfortably on a laptop. The justification for Spark lies in the scope of the analyses performed here: zone efficiency aggregations over all 4.2 million shots, a 21-season time-series, a binary classifier trained on the full record pool, and PCA plus K-Means over all qualifying player-seasons. Spark's lazy evaluation model allows these operations to be expressed as logical plans and executed in a single distributed pass, which is the appropriate tool for cross-era analyses that must treat all seasons as one coherent dataset.

2.4 Known Limitations

Three data limitations are relevant to interpretation. First, the dataset contains no defender or contest information; the shot-make model therefore measures the value of shot *location and action type*, not the quality of shot selection under defensive pressure. Second, the 2012 season was shortened by a labor lockout to 66 games (versus the standard 82), and the 2020 season was shortened and played in a bubble environment; both seasons show lower total attempt counts in Table 2 and should not be interpreted as trend deviations. Third, cross-era rule changes — most notably the 2004–05 defensive hand-check rule tightening — may affect FG% comparisons between early and late seasons in this panel.

3 Methodology

3.1 Zone Efficiency

Zone efficiency was computed by grouping all shots by `BASIC_ZONE` and calculating attempt count, field-goal percentage (FG%), and points per attempt (pts/attempt = FG% × SHOT_VALUE). Points per attempt is the theoretically correct efficiency metric because it accounts for the value differential between two- and three-point shots, enabling a fair comparison: a 33.3% three-point shooter and a 50% two-point shooter are equally efficient at 1.0 pts/attempt. Six zones are present after backcourt removal: Restricted Area, In The Paint (Non-RA), Mid-Range, Left Corner 3, Right Corner 3, and Above the Break 3.

3.2 Era Analysis

Three-point attempt rate (3PT attempts / total attempts × 100) and overall FG% were aggregated by season across all 21 seasons. The resulting time series (Table 2 and Figure 1) spans the pre-analytics era through peak three-point adoption, capturing the full arc from 2004 to 2024.

3.3 Shot-Make Model

A binary classification model predicts whether a given shot is made (`SHOT_MADE_INT = 1`). Three columns were excluded from the feature vector as leakage: `EXPECTED_PTS` (equals zero exactly when a shot is missed), `EVENT_TYPE` ("Made Shot"/"Missed Shot"), and `SHOT_MADE` (the boolean target itself).

The **feature vector** (26 dimensions) comprises eight numeric features — `LOC_X`, `LOC_Y`, `SHOT_DISTANCE`, `SECS_REMAINING`, `IS_3PT`, `SHOT_VALUE`, `IS_CLUTCH`, `QUARTER` — and four categorical features encoded via `StringIndexer` + `OneHotEncoder`: `BASIC_ZONE`, `SHOT_TYPE`, `ZONE_RANGE`, and `ACTION_GROUP`. All transformers were fit on the 80% training split and applied to the held-out 20% test split (random seed 42) to prevent information leakage. `StringIndexer` used `handleInvalid="keep"` so categories appearing only in the test set are assigned a reserved bucket rather than crashing the job.

LASSO regularization path. Feature importance was assessed by sweeping `regParam` over [0.05, 0.01, 0.005, 0.001, 0.0005, 0.0001] with `elasticNetParam = 1.0` (pure L1 penalty), yielding active feature counts of $3 \to 5 \to 9 \to 16 \to 21 \to 26$ as the penalty relaxes. The order in which features re-enter the model as regularization loosens provides a collinearity-robust importance ranking. Headline coefficients are reported at `regParam = 0.001` (16 of 26 features active), chosen as the penalty that preserves the zone structure without selecting all features back in.

Classifiers. `LogisticRegression` (L2, `regParam = 0.01`) and `LinearSVC` (`regParam = 0.01`) were trained on the same feature vector and evaluated on the held-out test split using ROC-AUC and macro-F1. All analyses used the full dataset (`SAMPLE_FRACTION = None`), encompassing all 4,222,442 records.

3.4 Player Archetypes

Player archetypes were identified by clustering player-season shot profiles. Each qualifying player-season was represented as a six-dimensional vector of zone shot shares (rim, paint, mid-range, left corner 3, right corner 3, above-the-break 3), which sum to one by construction. This compositional structure introduces a redundant direction (one share is determined by the other five), motivating PCA before clustering.

The pipeline applied: (1) **StandardScaler** (zero mean, unit variance per dimension, to prevent high-variance zones from dominating PCA); (2) **PCA** retaining 5 of 6 components (the sixth captures the near-zero compositional redundancy and is dropped); (3) **K-Means** at $k = 5$ (random seed 42), chosen based on silhouette score and interpretability of the resulting profiles. Era stability was assessed by holding cluster assignments fixed and comparing the share of player-seasons in each archetype for seasons before 2015 (`SEASON_1` < 2015) versus 2015 and after (`SEASON_1` 2015), with 2015 chosen as the inflection point identified in the era analysis (Section 4.2).

4 Results

4.1 Zone Efficiency: The Mid-Range Penalty

Table 1 reports field-goal percentage and points per attempt for each of the six shot zones across all 4,222,442 shots in the dataset.

Table 1: Zone efficiency across 4,222,442 shots (2004–2024). Points per attempt is field-goal percentage multiplied by shot value (2 or 3), the appropriate cross-zone efficiency metric. Zones are sorted by descending efficiency.

Zone	Attempts	FG%	Pts/Attempt
Restricted Area	1,344,005	61.6%	1.232
Right Corner 3	146,724	38.8%	1.164
Left Corner 3	159,656	38.6%	1.159
Above the Break 3	890,037	35.1%	1.054
In The Paint (Non-RA)	644,179	40.9%	0.818
Mid-Range	1,037,841	39.9%	0.798

4.2 Era Inflection: When the League Changed

Table 2 shows three-point attempt rate and overall FG% for every season from 2004 to 2024. Figure 1 plots the three-point rate as a time series.

Table 2: Three-point attempt rate and overall field-goal percentage by season (2004–2024). The 2012 season was lockout-shortened to 66 games; the 2020 season was played in a bubble environment. Both explain the lower attempt totals in those rows.

Season	Total Attempts	3PT Attempts	3PT Rate	FG%
2004	189,467	35,159	18.6%	43.9%
2005	197,263	38,386	19.5%	44.8%
2006	193,943	38,943	20.1%	45.5%
2007	195,696	41,296	21.1%	45.9%
2008	200,033	44,080	22.0%	45.8%
2009	198,585	44,117	22.2%	46.0%
2010	200,491	44,124	22.0%	46.2%
2011	199,292	43,815	22.0%	46.0%
2012	160,901	36,071	22.4%	44.9%
2013	201,159	48,617	24.2%	45.4%
2014	203,682	52,484	25.8%	45.5%
2015	205,123	54,690	26.7%	45.0%
2016	207,453	58,645	28.3%	45.3%
2017	209,430	65,739	31.4%	45.8%
2018	211,183	70,815	33.5%	46.1%
2019	218,992	78,276	35.7%	46.1%
2020	187,711	71,847	38.3%	46.1%

Table 2: Three-point attempt rate and overall field-goal percentage by season (2004–2024). The 2012 season was lockout-shortened to 66 games; the 2020 season was played in a bubble environment. Both explain the lower attempt totals in those rows.

Season	Total Attempts	3PT Attempts	3PT Rate	FG%
2021	190,674	74,513	39.1%	46.7%
2022	216,282	86,095	39.8%	46.2%
2023	216,814	83,758	38.6%	47.6%
2024	218,268	85,922	39.4%	47.5%

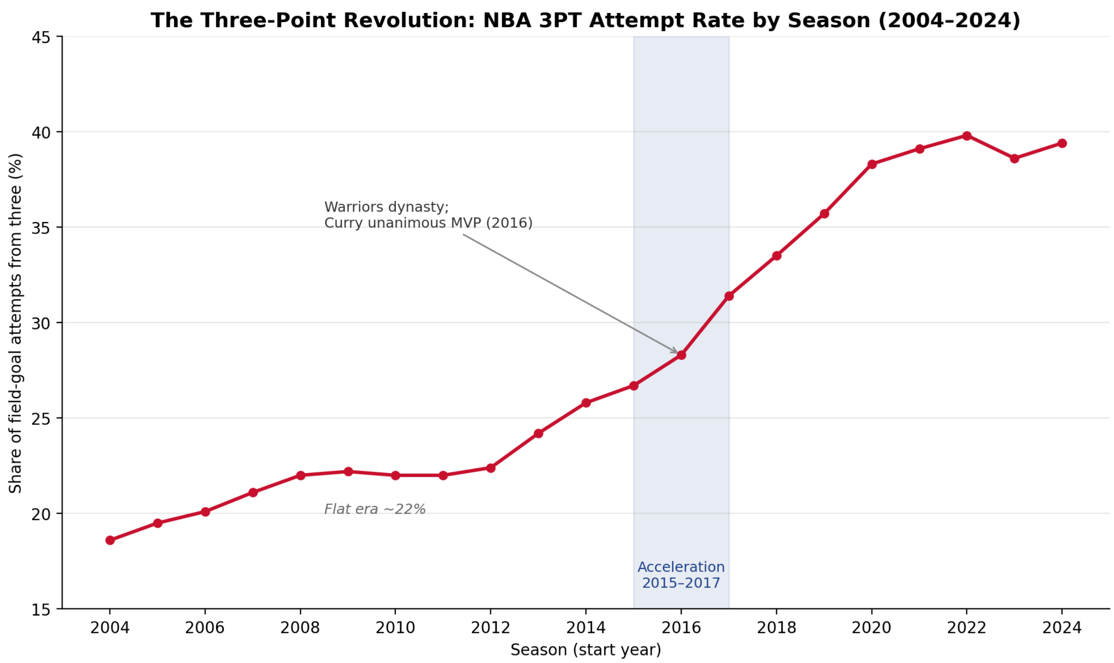

Figure 1: Three-point attempt rate by season, 2004–2024. The rate sat near 22% for approximately six seasons (2008–2013), then accelerated sharply from 26.7% in 2015 to 31.4% in 2017, and continued rising to a plateau of approximately 38–40% from 2020 onward.

4.3 Shot-Make Model: Location Over Context

Figure 2 shows the LASSO regularization path. Table 3 reports the headline coefficients at `regParam = 0.001`. Table 4 reports classifier performance on the held-out test split.

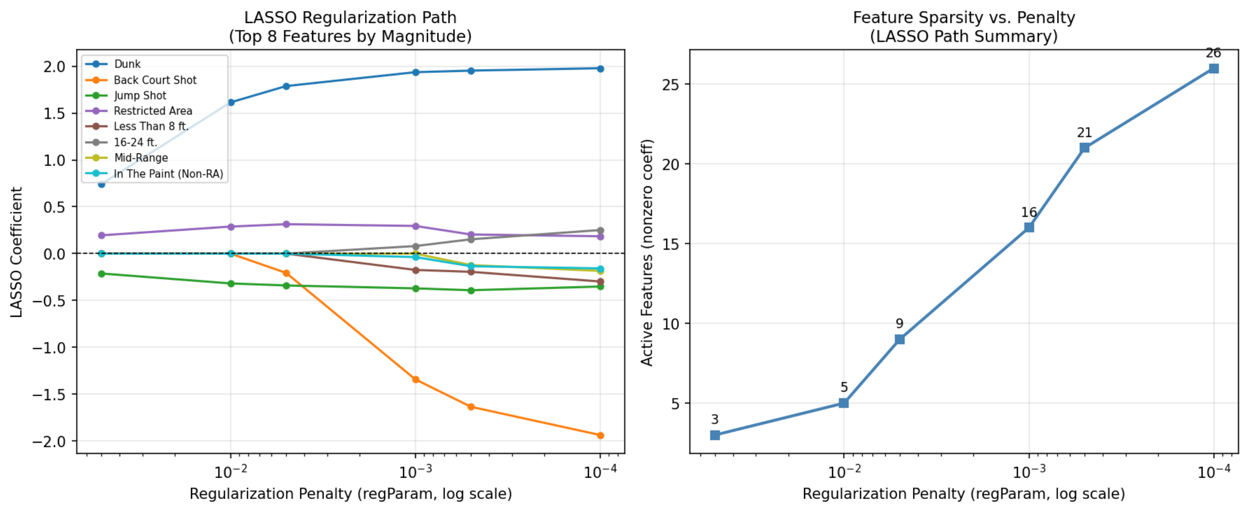

Figure 2: LASSO regularization path. Left panel: coefficient trajectories for the eight features with the largest peak magnitude as the penalty (regParam) relaxes from left to right. Right panel: number of active (nonzero) features at each penalty level. Context features (QUARTER, IS_CLUTCH, SECS_REMAINING) enter the model only at the weakest penalties and carry small coefficients throughout.

Table 3: LASSO coefficients at regParam $= 0.001$ (16 of 26 features active), sorted by absolute value. Positive values indicate higher make probability; negative values indicate lower. Features zeroed by LASSO at this penalty level are omitted. The Back Court Shot coefficient (-1.342) reflects a small residual of backcourt records that survived Phase 2 cleaning and does not affect any substantive conclusion.

Feature	Coefficient
Action: Dunk	1.9358
Range: Back Court Shot	-1.342
Action: Jump Shot	-0.3708
Zone: Restricted Area	0.2953
Range: Less Than 8 ft.	-0.1739
Range: 16-24 ft.	0.0805
Zone: Left Corner 3	0.0793
Zone: Right Corner 3	0.0791
Clutch Situation (Q4/OT, 120 s)	-0.0549
Action: Other	-0.0518
Zone: In The Paint (Non-RA)	-0.037
Quarter	-0.0171
Shot Distance (ft)	-0.015
Action: Hook	0.013
Court Y Coordinate	-0.0004
Seconds Remaining	0.0001

Table 4: Binary classifier performance on the held-out 20% test split (full dataset, random seed 42). The always-guess-miss accuracy floor is approximately 54% (league-wide make rate 46%). Both models comfortably exceed this floor; their near-identical performance corroborates the result independently of model choice.

Model	ROC-AUC	F1	Accuracy
LogisticRegression	0.629	0.605	0.618
LinearSVC	0.62	0.601	0.611

4.4 Player Archetypes: Who Thrived After the Revolution

Table 5 describes the five archetypes identified by K-Means clustering of 6,451 qualifying player-seasons. Table 6 compares archetype prevalence before and after the 2015 inflection point. Figure 3 shows the PCA scatter of all player-seasons colored by cluster. Figure 4 shows the before/after population breakdown by archetype.

Table 5: K-Means cluster shot profiles (k = 5) across 6,451 qualifying player-seasons (2004–2024). Zone shares are the fraction of a player-season's shots taken from each zone; rows sum to approximately 1.00. Clusters are labeled based on their dominant zone concentration. See Figure 3 for the PCA scatter plot.

Archetype	N	Rim	Paint	Mid-Range	L-Corner 3	R-Corner 3	ATB 3
Modern Versatile Scorer	1,422	31.3%	19.1%	15.6%	3.8%	3.5%	26.6%
Rim-Dominant Big	1,136	59.3%	22.6%	14.6%	0.5%	0.5%	2.4%
Floor-Spacing Specialist	866	24.2%	8.6%	12.4%	11.6%	11.0%	32.3%
High-Volume Perimeter	1,581	21.3%	9.3%	25.1%	5.4%	4.9%	34.0%
Mid-Range Specialist	1,446	31.1%	14.8%	42.7%	1.5%	1.4%	8.6%

Table 6: Archetype prevalence before and after the 2015 inflection point. Pre-2015: seasons 2004–2014 (3,207 player-seasons). Post-2015: seasons 2015–2024 (3,244 player-seasons). Δ is the change in percentage-point share of the qualifying player-season pool.

Archetype	Pre (n)	Pre (%)	Post (n)	Post (%)	Δ
Modern Versatile Scorer	260	8.1%	1,162	35.8%	+27.7 pp
Rim-Dominant Big	602	18.8%	534	16.5%	-2.3 pp
Floor-Spacing Specialist	276	8.6%	590	18.2%	+9.6 pp
High-Volume Perimeter	873	27.2%	708	21.8%	-5.4 pp
Mid-Range Specialist	1,196	37.3%	250	7.7%	-29.6 pp

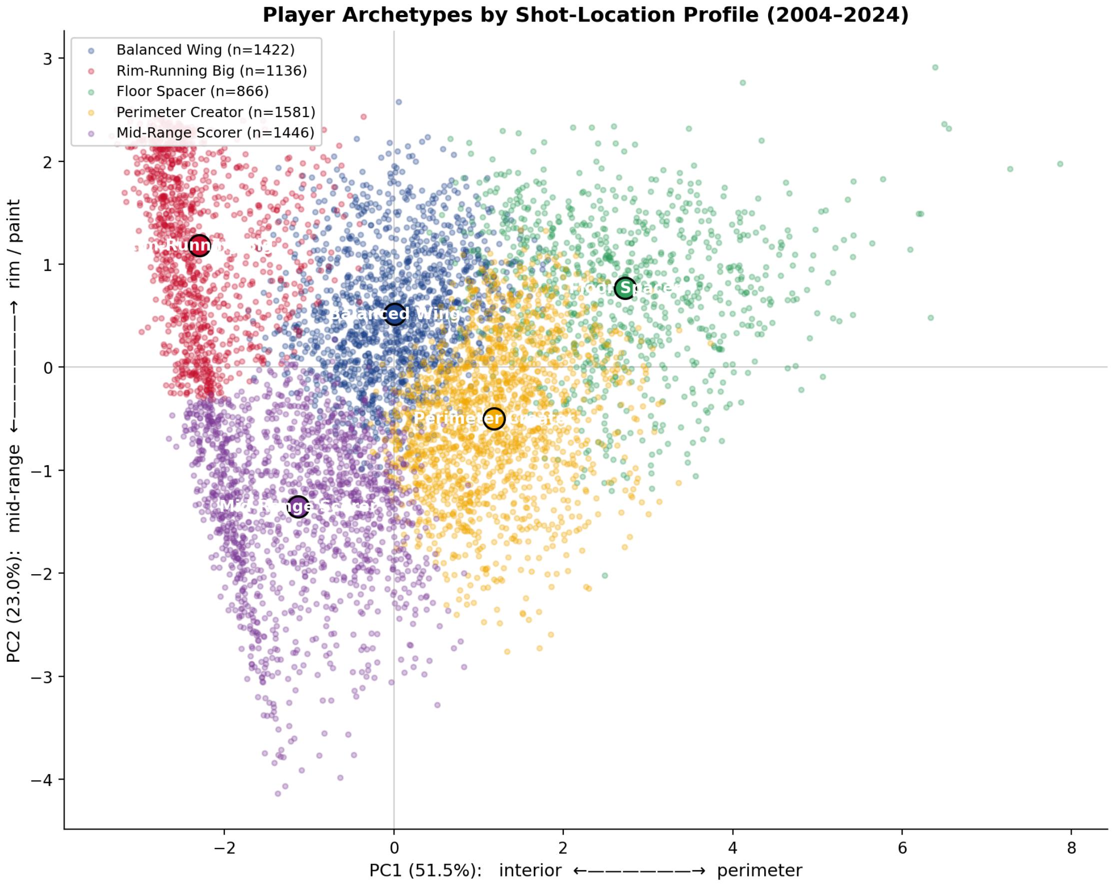

Figure 3: PCA scatter of 6,451 player-seasons colored by K-Means cluster (k = 5). PC1 and PC2 are the two dominant axes of shot-selection variation. Cluster separation reflects distinct shot profiles; see Table 5 for the zone-share interpretation of each archetype.

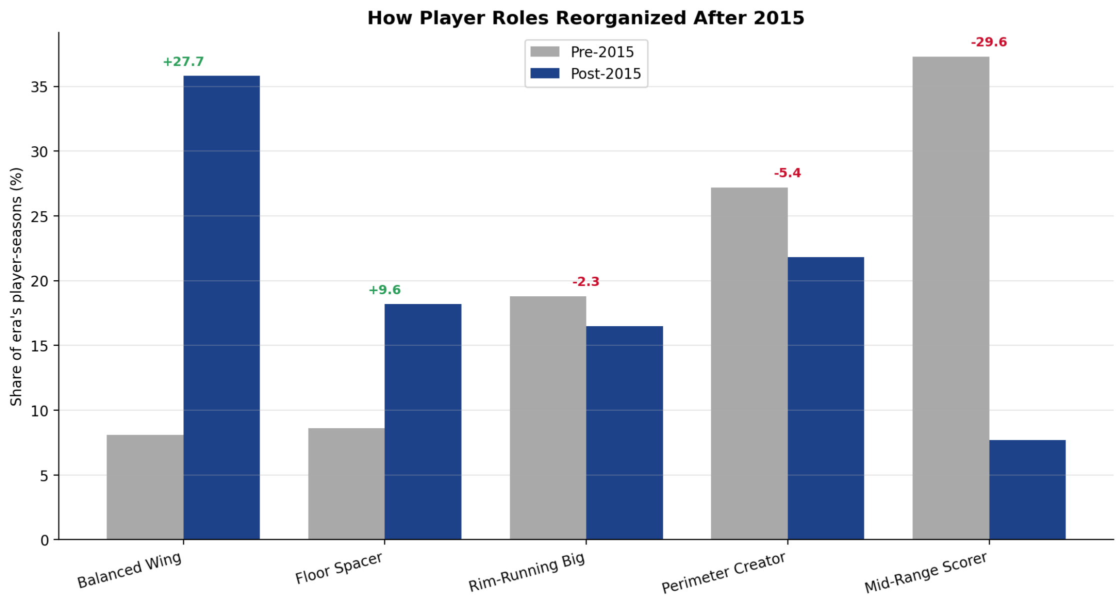

Figure 4: Archetype population shift before and after 2015. Each bar shows the percentage of qualifying player-seasons assigned to that archetype in the pre-2015 (seasons 2004–2014) and post-2015 (seasons 2015–2024) periods. The Mid-Range Specialist archetype declined 29.6 percentage points; the Modern Versatile Scorer grew 27.7 percentage points.

5 Conclusion and Future Study

6 References

Samangy, D. (2025). *NBA_Shots_04_25* [Data set]. GitHub. https://github.com/DomSamangy/NBA_Shots_04

National Basketball Association. (2025). *NBA Stats API*. https://www.stats.nba.com

Mexwell. (2024). *NBA shots* [Data set]. Kaggle. https://www.kaggle.com/datasets/mexwell/nba-shots

Apache Software Foundation. (2025). *Apache Spark documentation* (Version 3.5). https://spark.apache.org/docs/la

Apache Software Foundation. (2025). *MLlib: Machine learning library guide*. https://spark.apache.org/docs/latest guide.html

Google Cloud. (2025). *Dataproc documentation*. https://cloud.google.com/dataproc/docs

Appendix: Representative Code

The following excerpts are representative PySpark segments from the analysis scripts committed to the project repository. Full scripts are available in the repository at `02_processing/process_shots.py`, `03_shot_model/shot_model.py`, `03_shot_model/feature_engineering.py`, and `05_clustering/archetypes.p`

A.1 Zone Efficiency Aggregation (`02_processing/process_shots.py`)

```
# Aggregate zone efficiency: FG% and points per attempt by BASIC_ZONE
zone_stats = (
    shots
    .groupBy("BASIC_ZONE")
    .agg(
        F.count("*").alias("attempts"),
        F.round(F.avg("SHOT_MADE_INT") * 100, 1).alias("fg_pct"),
        F.round(F.avg("EXPECTED_PTS"), 3).alias("pts_per_attempt"),
    )
    .orderBy(F.desc("pts_per_attempt"))
)
zone_stats.write.csv(TABLES_PATH + "fg_pct_by_zone", header=True, mode="overwrite")
```

A.2 LASSO Regularization Path (`03_shot_model/shot_model.py`)

```
# Sweep regParam values and record the coefficient vector at each penalty
REG_PATH = [0.05, 0.01, 0.005, 0.001, 0.0005, 0.0001]
HEADLINE_REG = 0.001

path_rows = []
for reg in REG_PATH:
    lasso = LogisticRegression(
        featuresCol="features", labelCol=LABEL_COL,
        elasticNetParam=1.0,    # pure L1 = LASSO
        regParam=reg, maxIter=100,
    )
    pipeline = Pipeline(stages=build_feature_stages() + [lasso])
    model = pipeline.fit(train)
    lr_model = model.stages[-1]
    coeffs = lr_model.coefficients.toArray()
    feat_names = extract_feature_names(model.transform(train.limit(1)))
    for name, coef in zip(feat_names, coeffs):
        path_rows.append((reg, name, float(coef)))

path_df = spark.createDataFrame(path_rows, ["regParam", "feature", "coefficient"])
path_df.write.csv(LASSO_PATH_OUT, header=True, mode="overwrite")
```

A.3 PCA and K-Means Archetype Pipeline (`05_clustering/archetypes.py`)

```python
# StandardScaler → PCA (5 components) → K-Means (k=5, seed=42)
N_PCA = 5
CHOSEN_K = 5
SEED = 42

scaler = StandardScaler(inputCol="features", outputCol="scaled",
                        withMean=True, withStd=True)
players = scaler.fit(players).transform(players)

pca_model = PCA(k=N_PCA, inputCol="scaled", outputCol="pca_features").fit(players)
players = pca_model.transform(players).cache()

# Report explained variance per component
ev = pca_model.explainedVariance.toArray()
for j, frac in enumerate(ev):
    print(f"  PC{j+1}: {frac:.1%}  (cumulative {ev[:j+1].sum():.1%})")

# Final clustering
km_model = KMeans(featuresCol="pca_features", predictionCol="cluster",
                  k=CHOSEN_K, seed=SEED).fit(players)
assigned = km_model.transform(players)

# Era comparison: pre-2015 vs post-2015 cluster population shares
era_totals = {r["ERA"]: r["count"]
              for r in assigned.groupBy("ERA").count().collect()}
era_counts = (
    assigned
    .groupBy("cluster", "ERA")
    .count()
    .withColumn("pct", F.round(F.col("count") / era_totals[F.col("ERA")] * 100, 1))
)
```

AI-Use Statement